

# Table of contents

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                      | <b>3</b>  |
| 1.1      | Qu'est-ce que la surveillance d'intégrité des fichiers ?                 | 3         |
| 1.2      | Présentation du projet L-Watcher                                         | 3         |
| 1.3      | Contexte de développement                                                | 4         |
| <b>2</b> | <b>Fondation Théorique et Utilité du Projet</b>                          | <b>5</b>  |
| 2.1      | Pourquoi surveiller l'intégrité d'un système de fichiers ?               | 5         |
| 2.2      | Objectifs et cahier des charges de L-Watcher                             | 5         |
| 2.3      | L'approche hybride : C et Bash                                           | 6         |
| <b>3</b> | <b>Architecture et Structure du Programme</b>                            | <b>7</b>  |
| 3.1      | Modèle architectural à trois couches                                     | 7         |
| 3.2      | Cartographie de l'arborescence du projet (version installé)              | 8         |
| 3.2.1    | Le fichier de configuration                                              | 8         |
| 3.2.2    | Le Makefile                                                              | 9         |
| <b>4</b> | <b>L'API Linux Inotify au Cœur du Système</b>                            | <b>10</b> |
| 4.1      | Principes fondamentaux d'inotify                                         | 10        |
| 4.1.1    | La limite structurelle : pas de récursivité automatique                  | 10        |
| 4.1.2    | Le comportement du couple IN_CREATE / IN_ATTRIB                          | 10        |
| 4.2      | Implémentation en Langage C                                              | 11        |
| 4.2.1    | Lecture du tampon d'événements par arithmétique de pointeurs             | 11        |
| 4.2.2    | La table de correspondance watch descriptor → chemin                     | 11        |
| 4.2.3    | L'algorithme watch_recursive                                             | 12        |
| 4.2.4    | Le rôle critique de fflush(stdout)                                       | 12        |
| <b>5</b> | <b>Implémentation des Modules d'Automatisation</b>                       | <b>13</b> |
| 5.1      | Le pipeline d'affichage, de journalisation et de comparaison (output.sh) | 13        |
| 5.1.1    | Parsing du flux entrant                                                  | 13        |
| 5.1.2    | Filtrage de l'ATTRIB redondant                                           | 14        |
| 5.1.3    | Le mécanisme de différenciation                                          | 14        |
| 5.1.4    | Double écriture : console et log                                         | 15        |
| 5.2      | Snapshot de référence et historisation (backup.sh)                       | 15        |
| 5.2.1    | Constitution de la snapshot                                              | 15        |
| 5.2.2    | Archivage et transfert distant                                           | 15        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 5.3      | Cycle de vie des données et maintenance (cleanup.sh)      | 16        |
| 5.3.1    | Rotation du journal courant                               | 16        |
| 5.3.2    | Purge automatique des données obsolètes                   | 16        |
| <b>6</b> | <b>Guide d'Utilisation, Déploiement et Administration</b> | <b>18</b> |
| 6.1      | Prérequis et installation                                 | 18        |
| 6.2      | Configuration des répertoires surveillés                  | 18        |
| 6.3      | Maîtrise de l'interface en ligne de commande              | 19        |
| 6.3.1    | Exemples d'utilisation                                    | 19        |
| 6.4      | Lecture et interprétation des journaux de session         | 20        |
| 6.5      | Utilisation des fichiers de différences                   | 20        |
| <b>7</b> | <b>Evolution de la Conception et Pivot Technique</b>      | <b>22</b> |
| 7.1      | L'ambition initiale : le traçage d'acteur                 | 22        |
| 7.2      | Les approches testées et leurs limitations                | 22        |
| 7.2.1    | Tentative 1 : inspection de /proc/<PID>/fd                | 22        |
| 7.2.2    | Tentative 2 : lsof                                        | 23        |
| 7.2.3    | Tentative 3 : auditd et ausearch                          | 23        |
| 7.2.4    | Tentative 4 : variables d'environnement et who            | 23        |
| 7.3      | La décision de pivot                                      | 23        |
| <b>8</b> | <b>Perspective et développement au future</b>             | <b>25</b> |
| 8.1      | Bilan technique du projet L-Watcher                       | 25        |
| 8.1.1    | Forces du système                                         | 25        |
| 8.1.2    | Limites assumées (choix de conception)                    | 25        |
| 8.1.3    | Limitation résiduelle identifiée                          | 26        |
| 8.2      | Évolutions futures                                        | 26        |
| 8.2.1    | Interface utilisateur textuelle (TUI)                     | 26        |
| 8.2.2    | Dépassement du plafond de 32 répertoires                  | 26        |
| <b>9</b> | <b>Conclusion</b>                                         | <b>27</b> |

# 1 Introduction

## 1.1 Qu'est-ce que la surveillance d'intégrité des fichiers ?

Dans un système informatique en production, les fichiers ne sont jamais figés. Des configurations sont modifiées, des scripts sont mis à jour, des journaux sont écrits en continu. La plupart de ces changements sont légitimes et attendus. Certains ne le sont pas.

La surveillance d'intégrité des fichiers, connue sous l'acronyme **FIM** (*File Integrity Monitoring*), est la discipline qui consiste à détecter, horodater et documenter toute modification apportée à un système de fichiers. Son rôle est d'apporter une réponse concrète à une question simple : *quelque chose a changé, quoi, quand, et dans quelle mesure ?*

Les cas d'usage sont nombreux. Dans un contexte de sécurité, un FIM permet de détecter qu'un fichier de configuration critique a été altéré, qu'un binaire système a été remplacé, ou qu'un répertoire sensible a vu apparaître des fichiers inattendus. Dans un contexte d'exploitation, il permet de tracer l'historique précis des modifications apportées à des fichiers partagés, de reconstituer le fil des événements après une panne, ou simplement de maintenir un audit trail fiable.

## 1.2 Présentation du projet L-Watcher

**L-Watcher** est un outil FIM conçu pour Linux. Il surveille en temps réel un ou plusieurs répertoires configurés par l'utilisateur, détecte chaque événement survenu sur le système de fichiers comme création, modification, suppression, déplacement, changement de permissions d'un fichier et en produit une trace structurée et lisible.

Le projet repose sur deux technologies complémentaires. Le moteur de capture est écrit en **langage C** et exploite directement l'API noyau **inotify**, ce qui lui confère une réactivité maximale et une très faible empreinte système. Toute la couche de traitement, de mise en forme et de gestion du cycle de vie est prise en charge par des **scripts Bash**, qui assurent la production des journaux de session, la génération de fichiers de différences, la rotation des archives et les opérations de sauvegarde distante.

Le résultat est un système complet : d'un côté, un binaire C qui lit les événements noyau et les émet sur sa sortie standard ; de l'autre, une chaîne de scripts qui transforment ce flux brut en informations exploitables, persistées et maintenues dans le temps.

## 1.3 Contexte de développement

L-Watcher est né dans un contexte d'initiation aux systèmes Linux, avec l'objectif de construire quelque chose de réel et de fonctionnel plutôt qu'un exercice académique abstrait. Le développement a suivi une démarche pragmatique : chaque composant a été implémenté, testé en conditions réelles, puis affiné au contact des comportements effectifs du noyau et des limitations concrètes des outils disponibles. Certaines fonctionnalités initialement envisagées ont été abandonnées quand l'exploration technique a démontré leur infaisabilité dans ce contexte (ce parcours fait lui-même partie intégrante du projet, et sera documenté en détail dans la 8<sup>me</sup> partie.)

## 2 Fondation Théorique et Utilité du Projet

### 2.1 Pourquoi surveiller l'intégrité d'un système de fichiers ?

Un système Linux en fonctionnement est, par nature, un environnement dynamique. Des processus écrivent des fichiers en permanence, des droits d'accès évoluent, des répertoires apparaissent et disparaissent. Dans cet environnement mouvant, distinguer un changement attendu d'un changement anormal est difficile; surtout a posteriori, quand le problème est déjà survenu.

Les menaces qui pèsent sur l'intégrité d'un système de fichiers prennent deux grandes formes. La première est externe et malveillante : une intrusion qui modifie un binaire, un logiciel malveillant qui écrit des fichiers dans un répertoire système, un attaquant qui altère un fichier de configuration pour ouvrir une porte dérobée. La seconde est interne et souvent accidentelle : une mauvaise manipulation lors d'un déploiement, une modification non documentée d'un fichier partagé, une mise à jour qui écrase silencieusement une configuration personnalisée.

Dans les deux cas, la surveillance en temps réel offre un avantage décisif : elle permet de **détecter l'événement au moment où il se produit**, d'en conserver une trace horodatée, et grâce à la comparaison avec une version de référence de documenter précisément ce qui a changé. C'est la différence entre constater un dommage sans pouvoir en comprendre l'origine, et disposer d'un historique complet permettant de reconstituer exactement ce qui s'est passé.

### 2.2 Objectifs et cahier des charges de L-Watcher

L-Watcher a été conçu autour de deux objectifs fondamentaux.

Le premier est la **surveillance continue et récursive**. L'outil doit pouvoir surveiller non seulement un répertoire désigné, mais aussi l'intégralité de sa sous-arborescence y compris les nouveaux sous-répertoires créés en cours d'exécution. Aucune modification ne doit passer inaperçue, qu'elle survienne à la racine du chemin surveillé ou dans un répertoire imbriqué plusieurs niveaux plus bas.

Le second est la **production de rapports clairs et exploitables**. Il ne suffit pas de détecter qu'un événement a eu lieu : encore faut-il en produire une trace que l'on peut lire, analyser et archiver. L-Watcher génère à cet effet deux types de livrables. Les journaux de session (`/var/log/lwatcher/session_*.log`) consignent chaque événement avec son horodatage, son

type, le chemin concerné et les métadonnées associées (permissions, propriétaire, groupe). Les fichiers de différences (`/var/log/lwatcher/diffs/*.txt`) documentent, événement par événement, le contenu exact des modifications apportées aux fichiers surveillés.

## 2.3 L'approche hybride : C et Bash

Le choix de combiner C et Bash n'est pas arbitraire : il répond à une séparation naturelle des responsabilités.

Le langage C s'impose pour la couche de capture. L'API `inotify` est une interface noyau, et interagir avec elle de manière directe et efficace requiert la gestion fine de structures de données bas niveau, l'arithmétique de pointeurs pour parcourir le tampon d'événements, et la maîtrise des appels système. C'est exactement le domaine où C excelle. De plus, un binaire compilé démarré une seule fois et maintenu en attente dans une boucle bloquante sur `read()` a une empreinte système minimale.

Bash, en revanche, est parfaitement adapté à tout ce qui touche au formatage de texte, à la manipulation de fichiers, à l'invocation de commandes système (`stat`, `diff`, `find`, `tar`, `scp`) et à la gestion du cycle de vie des données. Écrire ces opérations en C serait inutilement verbeux et fragile là où quelques lignes de script suffisent.

La frontière entre les deux couches est très nette : le binaire C produit un flux de lignes texte formatées sur sa sortie standard, et les scripts Bash lisent ce flux via un pipe Unix classique. Cette interface minimale permet de faire évoluer chaque composant indépendamment.

## 3 Architecture et Structure du Programme

### 3.1 Modèle architectural à trois couches

L-Watcher s'articule autour de trois couches fonctionnelles clairement séparées, chacune assumant une responsabilité exclusive.

**La couche de capture** est incarnée par le binaire C (`bin/inotify`, compilé depuis `src/inotify.c`). Elle est en contact direct avec le noyau Linux via l'API `inotify`. Sa seule responsabilité est de recevoir les événements bruts, d'en extraire les informations essentielles (type d'événement, chemin du répertoire surveillé, nom du fichier concerné), et de les émettre sur la sortie standard sous forme de lignes texte délimitées par des deux-points :

```
EVENT:TYPE:CHEMIN:NOM
```

Par exemple : `MODIFIED:FILE:/home/user/docs/:rapport.txt`

Cette couche ne sait rien de la mise en forme, des couleurs, des logs, ou des sauvegardes. Elle produit un flux de données brut et s'arrête là.

**La couche de traitement et formatage** est assurée par `scripts/output.sh`. Elle lit le flux émis par le binaire C, l'interprète événement par événement, l'enrichit (horodatage, permissions via `stat`, comparaison via `diff`), l'affiche en console avec un formatage colorisé, et l'écrit simultanément dans les fichiers de log et de différences. C'est ici que réside l'essentiel de la logique métier de L-Watcher.

**La couche de maintenance et persistance** est portée par trois composants. `scripts/backup.sh` constitue au lancement la snapshot de référence qui servira de base aux comparaisons. `scripts/cleanup.sh` assure en fin de session la rotation des fichiers et la purge des données obsolètes. `lwatcher` (le script de lancement) orchestre l'ensemble : il traite les options en ligne de commande, prépare l'environnement, lance les autres composants dans le bon ordre, et installe le gestionnaire de signal qui déclenchera le nettoyage à la fermeture.

## 3.2 Cartographie de l'arborescence du projet (version installé)

```
/                                # Répertoire racine
usr/
  local/
    bin/
      lwatcher                  # Script de lancement placer dans $PATH

opt/
  lwatcher/
    bin/
      inotify                   # Binaire compilé (produit par make)
    scripts/
      output.sh                 # Traitement du flux, logs, diffs
      backup.sh                 # Snapshot de référence initiale
      cleanup.sh                # Rotation et purge en fin de session

etc/
  lwatcher/
    config/
      inotify.config            # Liste des répertoires à surveiller

var/
  log/
    lwatcher/
      session.log               # Journal courant (en cours de session)
      error.log                 # des erreurs
      session_YYYYMMDD_HHMM.log # Journaux archivés
      diffs/
        *.txt                   # Fichiers de différences par fichier modifié

root/
                                # Répertoire personnel de l'utilisateur root
  backup/
    SESSION_ID/                 # Snapshot de référence de la session (Répertoire
    SESSION_ID.tar.gz           # Archive compressée de la snapshot
```

### 3.2.1 Le fichier de configuration

/etc/lwatcher/config/inotify.config est le fichier qui définit quels répertoires L-Watcher doit surveiller. Sa syntaxe est volontairement minimaliste : un chemin absolu par ligne, les lignes vides et les lignes commençant par # étant ignorées. Ce fichier est lu à la fois par le binaire C (pour savoir quoi surveiller) et par `backup.sh` (pour savoir quoi sauvegarder).



```
# Répertoires surveillés par L-Watcher
/home/user/Public/
/etc/nginx/
```

### 3.2.2 Le Makefile

La compilation du binaire C est pilotée par un **Makefile** minimal :

```
all:
    gcc ./src/inotify.c -o ./bin/inotify

install:
    mkdir ~/lwatcher
    mkdir -p /opt/lwatcher
    mkdir -p /var/log/lwatcher
    mkdir -p /etc/lwatcher && touch /etc/lwatcher/inotify.config
    cp -r ./scripts /opt/lwatcher/ && cp -r ./bin /opt/lwatcher
    cp lwatcher /usr/local/bin
    chmod 755 /usr/local/bin/lwatcher
    chmod -R 755 /opt/lwatcher/*

remove:
    rm -rf /opt/lwatcher
    rm -f /usr/local/bin/lwatcher
    rm -rf /etc/lwatcher

purge:
    rm -rf /opt/lwatcher
    rm -rf /usr/local/bin/lwatcher
    rm -rf ~/lwatcher
    rm -rf /var/log/lwatcher
    rm -rf /etc/lwatcher

.PHONY: clean
clean:
    rm -f ./bin/inotify
```

## 4 L'API Linux Inotify au Cœur du Système

### 4.1 Principes fondamentaux d'inotify

`inotify` est une interface du noyau Linux qui permet à un processus de s'abonner aux événements survenant sur le système de fichiers. Contrairement à une approche par scrutation périodique où l'on vérifierait régulièrement l'état des fichiers pour détecter des changements, `inotify` fonctionne par **notification passive** : le processus se met en attente, et c'est le noyau lui-même qui l'informe dès qu'un événement se produit. Cette approche est à la fois plus réactive et moins coûteuse en ressources.

Le fonctionnement repose sur trois primitives principales. `inotify_init()` crée une instance `inotify` et retourne un descripteur de fichier. `inotify_add_watch()` associe à cette instance un répertoire ou fichier à surveiller, en précisant quels types d'événements on souhaite recevoir (créations, suppressions, modifications, changements de permissions, etc.). Enfin, un simple `read()` sur le descripteur `inotify` retourne un ou plusieurs événements sous forme de structures `inotify_event`, qui contiennent notamment un masque de bits décrivant la nature de l'événement et le nom du fichier concerné.

#### 4.1.1 La limite structurelle : pas de récursivité automatique

`inotify` surveille un répertoire donné, mais **pas** son contenu récursif. Si l'on ajoute `/home/user/docs/` à la surveillance, les événements sur les fichiers directement dans ce répertoire seront bien remontés, mais pas ceux qui surviendront dans `/home/user/docs/projets/` ou dans tout autre sous-répertoire. Cette limitation est fondamentale et impose d'implémenter soi-même la récursivité; ce que fait L-Watcher via la fonction `watch_recursive()`, décrite à la section suivante.

#### 4.1.2 Le comportement du couple `IN_CREATE` / `IN_ATTRIB`

Un point de comportement noyau mérite une attention particulière. Lorsqu'un nouveau fichier est créé, le noyau génère systématiquement **deux événements distincts et séquentiels** : d'abord `IN_CREATE`, qui signale l'apparition du fichier, puis `IN_ATTRIB`, qui signale que ses attributs (permissions, propriétaire) ont été définis. Ces deux événements sont inévitables et ne peuvent pas être distingués au niveau du masque de l'appel `inotify_add_watch()`.

Si l'on affiche naïvement tous les événements `ATTRIB` reçus, chaque création de fichier génère donc un faux positif. La solution adoptée dans L-Watcher est de filtrer cet `ATTRIB` redondant en aval, dans le script de traitement `output.sh`, en mémorisant le nom du dernier fichier créé et en supprimant le premier `ATTRIB` qui le concerne. Ce mécanisme est détaillé au chapitre 5.

## 4.2 Implémentation en Langage C

### 4.2.1 Lecture du tampon d'événements par arithmétique de pointeurs

La structure retournée par `read()` sur un descripteur inotify n'est pas un tableau d'éléments de taille fixe. Chaque événement est une `struct inotify_event` de taille variable, car le champ `name` : le nom du fichier concerné est de longueur variable et suivi d'un éventuel padding. Pour parcourir correctement le tampon, il faut donc avancer manuellement le pointeur de lecture, en ajoutant à chaque itération la taille fixe de la structure plus la longueur effective du champ `name` :

```
ptr += EVENT_SIZE + event->len;
```

C'est une application directe de l'arithmétique de pointeurs en C : `ptr` est un `char *` qui pointe successivement sur chaque événement dans le buffer. Cette technique est imposée par la conception de l'API et constitue un exemple concret de la gestion mémoire bas niveau que seul C permet d'exprimer aussi directement.

### 4.2.2 La table de correspondance watch descriptor → chemin

Chaque appel à `inotify_add_watch()` retourne un **watch descriptor** (un entier), identifiant unique de la surveillance d'un répertoire particulier. Lorsqu'un événement arrive, il contient ce watch descriptor, mais pas le chemin complet du répertoire concerné. Pour pouvoir produire une sortie lisible, il faut donc maintenir une table de correspondance entre les watch descriptors et les chemins absolus.

Dans L-Watcher, cette table est implémentée via deux tableaux globaux parallèles :

```
int wd[32];
char wd_path[32][512];
int wd_count;
```

`wd[i]` contient le watch descriptor du *i*-ème répertoire surveillé, et `wd_path[i]` contient son chemin absolu. La taille fixe de 32 entrées constitue une **limite volontaire et assumée** : dans le cadre du projet, surveiller plus de 32 répertoires distincts simultanément n'est pas un cas

d'usage prioritaire, et une structure dynamique (liste chaînée, allocation avec `realloc`) aurait complexifié significativement le code sans bénéfice immédiat. Cette limite est documentée comme un choix de conception et non comme un oubli.

### 4.2.3 L'algorithme `watch_recursive`

La récursivité est assurée par la fonction `watch_recursive()`, qui fonctionne en deux temps. Elle commence par ajouter le répertoire passé en paramètre à la surveillance inotify et à l'enregistrer dans la table de correspondance. Puis elle ouvre ce répertoire avec `opendir()` et itère sur son contenu avec `readdir()` : pour chaque entrée de type `DT_DIR` (en excluant `.` et `..`), elle se rappelle récursivement avec le chemin complet du sous-répertoire.

```
void watch_recursive(const char *path) {
    wd[wd_count] = inotify_add_watch(fd, path, IN_ALL_EVENTS);
    snprintf(wd_path[wd_count], 512, "%s", path);
    wd_count++;
    // ... opendir / readdir / appel récursif sur DT_DIR
}
```

Cette récursivité s'effectue une fois au démarrage, pour l'arborescence existante. Mais le programme gère également le cas des **sous-répertoires créés en cours d'exécution** : lorsqu'un événement `IN_CREATE` | `IN_ISDIR` est détecté dans la boucle d'événements principale, `watch_recursive()` est appelée immédiatement sur ce nouveau répertoire, étendant dynamiquement la surveillance à la nouvelle branche de l'arborescence.

### 4.2.4 Le rôle critique de `fflush(stdout)`

Le binaire C et le script `output.sh` sont connectés par un pipe Unix (`.../bin/inotify | .../scripts/output.sh`). Par défaut, la sortie standard d'un programme C est bufférisée en mode bloc lorsqu'elle n'est pas connectée à un terminal, c'est-à-dire précisément dans le cas d'un pipe. Concrètement, cela signifie que les événements seraient retenus dans le buffer interne et n'atteindraient le script Bash qu'une fois ce buffer plein, introduisant un délai inacceptable pour un outil de surveillance en temps réel.

L'appel à `fflush(stdout)` après chaque `printf()` d'événement force le vidage immédiat du buffer, garantissant que chaque événement est transmis au script de traitement sans délai. C'est un détail d'implémentation discret mais absolument essentiel au bon fonctionnement de l'architecture en pipeline.

## 5 Implémentation des Modules d'Automatisation

### 5.1 Le pipeline d'affichage, de journalisation et de comparaison (output.sh)

`output.sh` est le composant le plus complexe de la couche Bash. Il reçoit sur son entrée standard le flux brut émis par le binaire C, et en assure le traitement complet : enrichissement, affichage colorisé en console, écriture dans les journaux de session, et génération des fichiers de différences.

#### 5.1.1 Parsing du flux entrant

La lecture du flux s'effectue via une boucle `while` avec `IFS=: read` qui découpe chaque ligne selon le séparateur `:` :

```
while IFS=: read event type path name; do
    ...
done
```

Les quatre champs ainsi extraits correspondent exactement au format produit par le binaire C : le type d'événement (`CREATED`, `MODIFIED`, `DELETED`, etc.), le type de cible (`FILE` ou `DIRECTORY`), le chemin du répertoire surveillé, et le nom du fichier ou sous-répertoire concerné.

##### 5.1.1.1 Enrichissement des événements

Pour chaque événement reçu, le script calcule l'horodatage courant et interroge le système via `stat` pour obtenir les permissions octales, le propriétaire et le groupe de la cible concernée. Cette interrogation est conditionnelle : pour les événements `DELETED` et `MOVED_FROM`, la cible n'existe plus au moment du traitement, et `stat` ne peut pas être appelé.

```
if [[ "$event" != "DELETED" && "$event" != "MOVED_FROM" ]]; then
    perms=$(stat -c "%a" "$target")
    owner=$(stat -c "%U" "$target")
    group=$(stat -c "%G" "$target")
fi
```

### 5.1.2 Filtrage de l'ATTRIB redondant

Comme expliqué au chapitre 2, chaque création de fichier génère systématiquement un événement ATTRIB qui lui fait immédiatement suite. Pour éviter d'afficher un faux positif à chaque création, le script maintient une variable `last_created` qui mémorise le nom du dernier fichier créé. Au moment de traiter un événement ATTRIB, si son nom correspond à `last_created`, l'événement est silencieusement supprimé et `last_created` est réinitialisée.

```
if [[ "$event" == "CREATED" ]]; then
    last_created=$name
    # ... affichage et log
elif [[ "$event" == "ATTRIB" ]]; then
    if [ "$name" == "$last_created" ]; then
        last_created="" # suppression silencieuse
    else
        # ... affichage et log normaux
    fi
fi
```

### 5.1.3 Le mécanisme de différenciation

Le cœur analytique de `output.sh` réside dans le traitement des événements MODIFIED. Lorsqu'un tel événement est reçu pour un fichier, le script recherche si une version de référence de ce fichier existe dans la snapshot de sauvegarde constituée au démarrage de la session.

Si une référence est trouvée, il calcule le diff entre cette version et la version courante du fichier sur le disque :

```
diff_output=$(diff "$backup_file" "/${clean_path}/${name}")
changed=$(echo "$diff_output" | grep -c "^[<>]")
```

Le nombre de lignes modifiées (`changed`) est affiché dans la console et consigné dans le journal de session. Le détail complet du diff est archivé dans un fichier dédié dans `.../log/lwatcher/diffs/`, nommé selon le chemin et la session pour garantir l'unicité.

Si aucune référence n'est disponible (fichier créé après le démarrage de la session, par exemple), l'événement est quand même journalisé avec la mention `[No backup found]`.

### 5.1.4 Double écriture : console et log

Chaque événement est affiché sur la console **et** écrit dans le fichier journal `.../log/lwatcher/session.log`. L'affichage console utilise des codes ANSI pour la couleur ; l'écriture dans le fichier utilise `printf` sans codes couleur pour garantir la lisibilité du fichier texte brut.

## 5.2 Snapshot de référence et historisation (backup.sh)

`backup.sh` est exécuté **une seule fois**, au démarrage de chaque session, avant que le binaire C ne commence à remonter des événements. Son rôle est de constituer une **image de référence** de l'état des répertoires surveillés au moment du lancement.

### 5.2.1 Constitution de la snapshot

Le script lit le fichier `inotify.config` et, pour chaque chemin configuré, copie récursivement le contenu vers un répertoire de backup horodaté :

```
dir_backup="$backup_dir$line"
mkdir -p "$(dirname "$dir_backup")"
cp -r "$line" "$dir_backup"
```

Un point d'implémentation important mérite d'être signalé. La construction du chemin de destination utilise `dirname` pour créer uniquement le répertoire parent, et non la destination finale elle-même. Cela évite un piège classique de `cp -r` : si la destination existe déjà au moment de la copie, `cp -r` l'imbrique à l'intérieur au lieu de la remplacer, produisant un niveau de nesting supplémentaire indésirable.

Cette snapshot est ensuite utilisée par `output.sh` comme base de comparaison pour tous les diffs de la session. Elle représente l'état "propre" connu du système de fichiers au moment où la surveillance a démarré.

### 5.2.2 Archivage et transfert distant

Une fois la copie effectuée, la snapshot est compressée dans une archive `tar.gz` horodatée :

```
tar -czf "/root/backup/$time_stamp.tar.gz" -C "/root/backup" "$time_stamp"
```

Si l'option de sauvegarde distante est activée (via le flag `-r`), l'archive est transmise vers le serveur distant via `scp`. La connexion s'appuie sur une paire de clés SSH dédiée (`lwatch_key`) générée spécifiquement pour la session par `lwatcher`, évitant l'utilisation des clés personnelles de l'utilisateur.

## 5.3 Cycle de vie des données et maintenance (`cleanup.sh`)

`cleanup.sh` est déclenché automatiquement en fin de session, via un `trap ... EXIT` installé par le script de lancement. Qu'il s'agisse d'une fermeture normale (Ctrl+C) ou d'un arrêt via signal `SIGTERM`, ce script garantit que les données sont toujours correctement rangées à la sortie du programme.

### 5.3.1 Rotation du journal courant

Pendant la session, les événements sont écrits dans un fichier temporaire `.../log/lwatcher/session.log`. En fin de session, ce fichier est renommé avec un horodatage permanent :

```
mv "$log_file" "./logs/session_$(date +%Y%m%d)_$(date +%H%M).log"
```

Cette rotation garantit que chaque session dispose de son propre fichier de journal immuable, et que le fichier `session.log` est toujours vide au démarrage d'une nouvelle session.

### 5.3.2 Purge automatique des données obsolètes

Le script supprime ensuite tous les fichiers plus anciens que la durée de rétention configurée (`max_days`, 7 jours par défaut) :

```
find "$log_dir" -name "session_*.log" -mtime "+$max_days" -delete
find "$backup_dir" -name "*.tar.gz" -mtime "+$max_days" -delete
find "$diff_dir" -name "*.txt" -mtime "+$max_days" -delete
```

La snapshot de la session courante (répertoire temporaire `backup/SESSION_ID/`) est également supprimée, seule l'archive `.tar.gz` correspondante étant conservée pour la durée de rétention.



### 5.3.2.1 Nettoyage des artefacts SSH

Si la fonctionnalité de sauvegarde distante était activée pour la session, `cleanup.sh` prend en charge la suppression de la clé SSH temporaire générée :

```
ssh -i ~/.ssh/lwatch_key "${remote_backup%%:*}" \  
    "sed -i '/lwatch_key/d' ~/.ssh/authorized_keys" \  
rm -f ~/.ssh/lwatch_key ~/.ssh/lwatch_key.pub
```

Cette étape retire d'abord l'entrée correspondante dans le fichier `authorized_keys` du serveur distant (via une commande SSH), puis supprime la paire de clés localement. L'objectif est de limiter la durée d'exposition de ces credentials temporaires au strict minimum nécessaire.

# 6 Guide d'Utilisation, Déploiement et Administration

## 6.1 Prérequis et installation

L-Watcher nécessite un système Linux avec les outils suivants disponibles : `gcc` pour la compilation du binaire C, `make` pour piloter la compilation, et les utilitaires standard `bash`, `stat`, `diff`, `find`, `tar`. Pour la fonctionnalité de sauvegarde distante, `ssh`, `ssh-keygen`, `ssh-copy-id` et `scp` sont également requis.

- L'installation s'effectue simplement avec :

```
$ make          # compile src/inotify.c
$ sudo make install
    # installe /opt/lwatcher, /etc/lwatcher,
    # /var/log/lwatcher, /usr/local/bin/lwatcher
$ which lwatcher    # vérification
```

- Pour la désinstallation et la suppression :

```
$ sudo make remove    # garde les log et backups
$ sudo make purge     # supprime tout
```

## 6.2 Configuration des répertoires surveillés

Avant le premier lancement, il faut renseigner le fichier `config/inotify.config` avec les chemins à surveiller. Sa syntaxe est très simple :

```
# Les lignes commençant par # sont ignorées
# Les lignes vides sont ignorées

/home/user/Documents/
/etc/nginx/conf.d/
```

Chaque chemin doit être absolu et se terminer par /. Plusieurs chemins peuvent être ajoutés, chacun sur sa propre ligne.

Les options `-c`, `-C` et `-a` de `start.sh` permettent de manipuler ce fichier directement en ligne de commande, sans avoir à l'éditer manuellement.

## 6.3 Maîtrise de l'interface en ligne de commande

Toutes les interactions avec L-Watcher passent par `start.sh`. Voici le panorama complet de ses options :

| Op-<br>tion     | Argument         | Comportement                                                   |
|-----------------|------------------|----------------------------------------------------------------|
| <code>-h</code> | —                | Affiche le manuel d'aide et quitte                             |
| <code>-s</code> | —                | Affiche la liste des répertoires surveillés et quitte          |
| <code>-D</code> | —                | Vide la liste des répertoires surveillés et quitte             |
| <code>-d</code> | N                | Définit la durée de rétention à N jours (défaut : 7)           |
| <code>-c</code> | CHEMIN           | Ajoute CHEMIN à la liste, puis lance la surveillance           |
| <code>-C</code> | CHEMIN           | Remplace toute la liste par CHEMIN, puis lance la surveillance |
| <code>-a</code> | CHEMIN           | Ajoute CHEMIN à la liste <b>sans</b> lancer la surveillance    |
| <code>-r</code> | USER@HOST:CHEMIN | Active la sauvegarde distante via SSH/SCP                      |

### 6.3.1 Exemples d'utilisation

Lancement simple avec la configuration existante :

```
lwatcher
```

Lancement avec une rétention de 14 jours :

```
lwatcher -d 14
```

Ajouter un répertoire et démarrer immédiatement :

```
lwatcher -c /home/user/projets/
```

Surveiller uniquement un répertoire spécifique (efface les chemins précédents) :

```
lwatcher -C /etc/
```

Ajouter un répertoire sans démarrer (pour préparer la configuration) :

```
lwatcher -a /var/www/html/
```

Lancement avec sauvegarde distante :

```
lwatcher -r admin@192.168.1.10:/backups/lwatcher
```

Combiner plusieurs options :

```
lwatcher -d 30 -r admin@192.168.1.10:/backups/lwatcher
```

**Note :** lwatcher doit être exécuter en tant que root pour fonctionner

## 6.4 Lecture et interprétation des journaux de session

Un journal de session se présente sous la forme d'un tableau aligné en colonnes :

|                       |          |      |                      |             |     |      |      |
|-----------------------|----------|------|----------------------|-------------|-----|------|------|
| [Mer 17 Jun 09:14:47] | CREATED  | FILE | /home/user/docs/okl/ | rapport.txt | 644 | user | user |
| [Mer 17 Jun 09:14:58] | MODIFIED | FILE | /home/user/docs/okl/ | rapport.txt | 644 | user | user |

Les colonnes, dans l'ordre, sont : l'horodatage, le type d'événement, le type de cible (FILE ou DIRECTORY), le chemin du répertoire, le nom du fichier, les permissions en octal, le propriétaire et le groupe. Pour les événements MODIFIED ayant une référence de backup, une indication du nombre de lignes modifiées et du chemin vers le diff complet est ajoutée en fin de ligne.

## 6.5 Utilisation des fichiers de différences

Les fichiers de différences se trouvent dans `.../log/diffs/`. Leur nom encode le chemin du répertoire (les `/` remplacés par `_`), le nom du fichier, et l'identifiant de session. Par exemple, pour un fichier `rapport.txt` dans `/home/user/docs/okl/` lors de la session `20260617_0934`, le fichier de diff se nomme :

```
home_user_docs_okl__rapport.txt_20260617_0934.txt
```

Son contenu liste chronologiquement chaque modification, avec l'horodatage et le diff au format standard Unix :

```
[Mer 17 Jun 09:14:58]    /home/user/docs/okl/    rapport.txt
1a2
> Nouvelle ligne ajoutée

[Mer 17 Jun 09:15:30]    /home/user/docs/okl/    rapport.txt
2c2
< Nouvelle ligne ajoutée
> Ligne modifiée
```

Le format de diff utilise les conventions standard : > pour une ligne ajoutée, < pour une ligne supprimée, c pour une ligne changée, a pour un ajout, d pour une suppression.

## 7 Evolution de la Conception et Pivot Technique

### 7.1 L'ambition initiale : le traçage d'acteur

Dans sa conception originale, L-Watcher devait répondre à une question plus ambitieuse que “qu’est-ce qui a changé ?” : il devait aussi répondre à “qui a fait ce changement ?”. L’idée était d’associer à chaque événement détecté l’identité de l’utilisateur ou du processus à l’origine de la modification, ce que l’on appellera le **traçage d’acteur** (*actor tracking*).

Cette fonctionnalité aurait considérablement enrichi la valeur analytique des journaux produits. Au lieu de voir simplement `MODIFIED: /etc/passwd`, un administrateur aurait pu lire `MODIFIED: /etc/passwd` par l'utilisateur root via le processus passwd (PID 4821). L’objectif était légitime et la question posée était pertinente.

Les tentatives d’implémentation ont cependant révélé une série d’obstacles techniques qui ont conduit à l’abandon définitif de cette fonctionnalité.

### 7.2 Les approches testées et leurs limitations

#### 7.2.1 Tentative 1 : inspection de `/proc/<PID>/fd`

La première approche consistait à scanner, au moment de la réception d’un événement inotify, le système de fichiers virtuel `/proc` pour trouver quel processus avait ouvert le fichier concerné. Pour chaque processus actif, on examine `/proc/<PID>/fd/` à la recherche d’un descripteur de fichier pointant vers le fichier cible.

En théorie, l’idée est solide. En pratique, elle se heurte à un problème fondamental de **timing**. Les opérations sur les fichiers, surtout les créations et les écritures brèves sont extrêmement rapides. Au moment où l’événement inotify est reçu et traité, le processus responsable a déjà terminé son opération, fermé son descripteur de fichier, et souvent terminé son exécution. Le scan de `/proc` arrive toujours trop tard.

### 7.2.2 Tentative 2 : lsof

`lsof` (*list open files*) est un outil qui interroge l'état courant des fichiers ouverts sur le système. On aurait pu l'invoquer à la réception de chaque événement pour identifier le processus concerné.

Outre le problème de timing déjà évoqué, cette approche souffre de deux défauts supplémentaires. D'abord, la latence d'exécution de `lsof` lui-même est non négligeable; le lancer à chaque événement aurait créé un goulot d'étranglement notable. Ensuite, `lsof` retourne l'état courant, pas l'état passé : si le fichier a été fermé avant l'invocation de la commande, il n'apparaît tout simplement plus dans les résultats.

### 7.2.3 Tentative 3 : auditd et ausearch

`auditd` est le démon d'audit du noyau Linux. Contrairement aux approches précédentes, il intercepte les appels système au moment où ils se produisent, ce qui résout le problème de timing. En théorie, il peut enregistrer précisément quel utilisateur et quel processus ont effectué chaque opération sur le système de fichiers.

L'implémentation a été tentée avec `auditctl` pour configurer des règles de surveillance et `ausearch` pour interroger les logs d'audit produits. Cette approche s'est avérée incompatible avec l'architecture en pipeline de L-Watcher. `ausearch` invoqué depuis l'intérieur du pipe ne retournait aucun résultat utilisable. Plus fondamentalement, intégrer `auditd` dans le flux de traitement posait des problèmes de permissions (l'outil requiert des droits root), de couplage architectural (L-Watcher devenait dépendant d'un daemon externe configuré séparément), et de complexité disproportionnée par rapport à l'objectif initial.

### 7.2.4 Tentative 4 : variables d'environnement et who

D'autres pistes ont été explorées : lire `/proc/<PID>/environ` pour récupérer des informations sur l'environnement du processus, utiliser `who` et `who am i` pour identifier l'utilisateur connecté. Aucune n'a abouti à quelque chose de fiable. `who` ne peut pas détecter les sessions ouvertes via `su`, et l'identification via les variables d'environnement reste sujette aux mêmes problèmes de timing.

## 7.3 La décision de pivot

Face à l'échec systématique de toutes les approches testées, la décision a été prise d'**abandonner définitivement le traçage d'acteur**. Cette décision n'est pas un compromis subi mais un choix raisonné.

Premièrement, les limitations constatées ne sont pas des bugs corrigeables : elles sont inhérentes à l'architecture de l'API inotify, qui notifie *après* l'événement, alors que l'identification d'un acteur requiert d'être *synchrone* avec lui. Rendre le traçage d'acteur fiable aurait nécessité une refonte complète de la couche de capture, vraisemblablement en abandonnant inotify au profit d'une approche eBPF ou d'une intégration profonde avec auditd; deux technologies qui dépassent largement le cadre du projet.

Deuxièmement, L-Watcher répond déjà à la question la plus utile dans la plupart des scénarios d'usage : **ce qui a changé, et quand**. Le scénario de démonstration typique: un fichier de configuration qui a été modifié de manière inexpliquée, est couvert par les journaux et les diffs. L'identification de l'auteur, bien que souhaitable, n'est pas indispensable à la valeur fondamentale de l'outil.

Ce parcours d'exploration et d'échec fait partie de l'histoire technique de L-Watcher et illustre une réalité du développement système : certaines fonctionnalités qui semblent simples en théorie se heurtent à des contraintes architecturales profondes qui rendent leur implémentation fiable bien plus complexe qu'anticipé.



## 8 Perspective et développement au future

### 8.1 Bilan technique du projet L-Watcher

L-Watcher remplit les objectifs qu'il s'était fixés. La surveillance est continue, réursive, et réactive. Chaque événement est capturé, enrichi, affiché et archivé. Les modifications de fichiers sont documentées au niveau du contenu, pas seulement de l'existence. Le cycle de vie des données est géré automatiquement.

#### 8.1.1 Forces du système

La principale force de L-Watcher est sa **légèreté architecturale**. L'API inotify est une interface noyau : il n'y a pas de scrutation périodique, pas de surcoût CPU en l'absence d'événements. Le binaire C se contente d'attendre sur un `read()` bloquant, et ne consomme des ressources que quand il y a effectivement quelque chose à traiter.

La **traçabilité fine par diffing** est la seconde force distinctive. La plupart des outils FIM simples se contentent de signaler qu'un fichier a changé ; L-Watcher documente précisément ce qui a changé, ligne par ligne, avec un historique cumulatif par fichier au sein d'une session.

L'**architecture modulaire**, la séparation nette entre le moteur C et les scripts Bash connectés par un pipe standard facilite la maintenance et l'évolution indépendante de chaque composant.

#### 8.1.2 Limites assumées (choix de conception)

Deux limitations sont des choix délibérés, documentés comme tels plutôt que dissimulés.

Le **plafond de 32 répertoires surveillés simultanément** découle de l'utilisation de tableaux de taille fixe pour la table de correspondance watch descriptor → chemin. Ce choix simplifie significativement le code et couvre largement les cas d'usage envisagés. Une implémentation future pourrait remplacer ces tableaux statiques par une structure dynamique (`realloc`) pour lever cette limite.

L'**absence de traçage d'acteur** est une conséquence directe des explorations techniques documentées au chapitre 4. Ce n'est pas une limitation oubliée mais une décision raisonnée, prise après avoir épuisé les approches réalistement implémentables dans ce contexte.

### 8.1.3 Limitation résiduelle identifiée

Un comportement imparfait subsiste pour les événements `ATTRIB` portant sur des répertoires. Dans ce cas, le calcul des permissions via `stat` est effectué sur le répertoire parent plutôt que sur le sous-répertoire effectivement concerné par l'événement. Ceci est dû à la manière dont le chemin de la cible est construit dans `output.sh` pour les événements de type `DIRECTORY`. Ce comportement erroné est identifié et pourrait faire l'objet d'un correctif dans une version ultérieure.

## 8.2 Évolutions futures

### 8.2.1 Interface utilisateur textuelle (TUI)

L'affichage actuel en colonnes fixes dans le terminal est fonctionnel mais statique. Une évolution naturelle serait de le remplacer par une interface TUI (*Text User Interface*) interactive, permettant par exemple de filtrer les événements affichés en temps réel par type ou par répertoire, de naviguer dans l'historique de la session, ou d'afficher le détail d'un diff directement dans l'interface. Des bibliothèques comme `ncurses` (pour une implémentation C) ou des outils comme `dialog` (pour du Bash) permettraient d'atteindre cet objectif.

### 8.2.2 Dépassement du plafond de 32 répertoires

Remplacer les tableaux statiques `wd[32]` et `wd_path[32]` [512] par des structures allouées dynamiquement permettrait de lever la limite actuelle sans autre modification architecturale. C'est une évolution relativement ciblée qui rendrait L-Watcher utilisable pour surveiller des arborescences bien plus profondes.

## 9 Conclusion

L-Watcher est un système FIM fonctionnel, documenté et maintenu. Il illustre concrètement comment combiner un composant C bas niveau avec des scripts shell pour produire un outil système complet, comment naviguer dans les limitations d'une API noyau, et comment les contraintes rencontrées en chemin peuvent être transformées en matière d'apprentissage et en décisions d'architecture explicites et défendables plutôt que d'être dissimulées .